Tut-8

1. a) True for undirected

no. of tree edges = $n - k$

n = no. of vertices

k = no. of ~~connected~~ components

no. of back edges = $m - (n - k)$

m = no. of edges

False for directed

b) True

All dfs forests have the same no. of trees = no. of connected components

2. a) False.

Dfs order depends on:

- starting vertex
- adjacency list order

b) True

all adjacent vertices were already visited before u.

c) True

Dfs visits u first then has path to v

$v.d \leq u.f$

Dfs visits v first ~~from~~ another path then v.d is already $\leq u.f$

$v.d \leq u.f$

3) check-hamiltonian-path-DAG(G):

```
topo = topological sort(G)
for i = 1 to |V|-1:
    if edge (topo[i], top[i+1]) not in
        return False
return TRUE
```

T.C = O(V+E) for topological sorting

T.C = O(V) for edge

T.C = O(V+E)

4) compute-cost(G, price)

```
topo = topological sort(G)
for each vertex u in V
    cost[u] = price[u]
for each vertex u in reverse(topo):
    for each v in adj[u]
        cost[u] = min(cost[u],
                      cost[v])
return cost
```

TC = O(V+E)

5) An undirected connected graph can be oriented to become a strongly connected directed graph iff it has no cut edges (Robbin's theorem)

```
check-strong-oriented (G):
    for each vertex u ∈ G.V:
        visited[u] = False
    time = 0
    for each vertex u ∈ G.V:
        if visited[u] == False
            if DFS(u, NIL) == TRUE
                return False
    return True

DFS(u, parent):
    visited[u] = true
    time += 1
    u.d = time
    h[u] = time
    for each v ∈ G.adj[u]
        if v == parent
            continue
        if visited[v] == false
            if DFS(v, u) == True
                return True
            h[u] = min{h[u], h[v]}
            if h[v] > u.d
                return True
        else
            time += 1
            h[u] = min{h[u], v.d}
    return false
```

6) a) strongly-connected(G):

DFS(G)

if some vertex not visited in G

return False

compute G^T

DFS(G^T)

if some vertex not visited in G^T

return False

return True.

b) Return-To-Airport(G, s):

R1 = DFS(G, s)

compute G^T

R2 = DFS(G^T, s)

for each vertex v in R1

if v not in R2

return False

return True

7) a) find-vebel(G):

visited[v] = False for all v

candidate = nil

for each vertex v in G.V

if visited[v] == False

DFS(v)

```
candidate = v
visited[v] = (all, false)
DFS(candidate)
if all vertices visited
        return candidate
else
        return "NO solution".
```

b) Min-Rebels(G):

```
compute SCCs of G
build condensed DAG
indegree[scc] = 0
for each edge(u,v)
    if scc(u) ≠ scc(v)
        indegree[scc(v)] ++
count = no. of SCCs with indegree = 0
return count
```